

METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE
Documento de Arquitetura de Software

CLOUD KEY

E-commerce de Keys Eletrônicas de Jogos Digitais
Documento de Arquitetura de Software (DAS)

CK – Cloud Key	
Gestor do Projeto	Gerente de Projeto
Grupo 8 — Curso de Engenharia/Ciência da Computação	Grupo 8 — Disciplina de Arquitetura de Software
Universidade Católica de Brasília (UCB)	Período letivo 2026/1

Objetivo deste Documento
Este documento tem como objetivo descrever as principais decisões de arquitetura tomadas pela equipe de desenvolvimento do sistema Cloud Key e os critérios considerados durante a tomada dessas decisões, abrangendo aspectos de software e de infraestrutura (hardware). A organização segue o modelo de visões “4+1” de Kruchten.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
07/04/2026	CK000001	Grupo 8	Elaboração inicial do Documento de Arquitetura de Software do Cloud Key.	1.0

Sumário

1. INTRODUÇÃO	3
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
2. REPRESENTAÇÃO ARQUITETURAL	6
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	8
4. VISÃO DE CASOS DE USO	10
4.1 Casos de Uso significantes para a arquitetura	10
5. VISÃO LÓGICA	12
5.1 Visão Geral — pacotes e camadas	12
6. VISÃO DE IMPLEMENTAÇÃO	15
6.1 Caso de Uso CU004 — Comprar key de jogo	15
6.1.1 Diagrama de Classes	15
6.1.2 Diagrama de Sequência	16
7. VISÃO DE IMPLANTAÇÃO	18
8. DIMENSIONAMENTO E PERFORMANCE	20
8.1 Volume	20
8.2 Performance	20
9. QUALIDADE	21
Guia para elaboração dos diagramas	22
1. Diagrama de Casos de Uso significativo	22
2. Diagrama de Pacotes / Camadas da Visão Lógica	22
3. Diagrama de Classes do CU004 — Comprar key de jogo	23
4. Diagrama de Sequência do CU004 — Comprar key de jogo	23
5. Diagrama de Implantação	24
6. MER / Diagrama Entidade-Relacionamento	24

1. INTRODUÇÃO

1.1 Finalidade

Este Documento de Arquitetura de Software (DAS) fornece uma visão arquitetural abrangente do sistema Cloud Key, um e-commerce de keys eletrônicas de jogos digitais desenvolvido pelo Grupo 8 como projeto acadêmico das disciplinas de Arquitetura de Software e Programação para Web da Universidade Católica de Brasília (UCB), no período letivo 2026/1. O documento tem por finalidade capturar, organizar e comunicar as decisões arquiteturais significativas tomadas em relação ao sistema, oferecendo a cada parte interessada — analistas, programadores, integradores, equipe de operações e responsáveis pela infraestrutura — uma perspectiva adequada às suas necessidades.

Para estruturar essa comunicação, o documento adota o modelo de quatro visões mais uma de Kruchten (modelo “4+1”), no qual as visões Lógica, de Processos, de Implementação e de Implantação são articuladas em torno de uma visão central de Casos de Uso. Dessa forma, o DAS serve como referência única para o desenvolvimento, a evolução e a manutenção do Cloud Key, garantindo que as escolhas técnicas estejam coerentes com as necessidades de negócio e com os requisitos não funcionais previamente consolidados no Documento de Visão do projeto.

1.2 Escopo

Este Documento de Arquitetura de Software aplica-se ao sistema Cloud Key, uma aplicação web multicliente voltada à compra e venda de chaves de ativação (keys) de jogos digitais, inspirada na plataforma Nuuvem e direcionada principalmente ao público brasileiro e latino-americano. O sistema atende jogadores de PC, consoles e dispositivos móveis que desejam adquirir jogos oficiais com segurança, bom preço e em português, bem como a equipe administrativa responsável pela gestão do catálogo, do estoque de chaves e dos pedidos.

No escopo arquitetural definido por este documento estão as capacidades centrais da plataforma: o cadastro e a autenticação de usuários, a navegação e a consulta ao catálogo de jogos, o fluxo de compra por meio de carrinho e checkout, a geração e a associação de chaves às contas dos clientes após a confirmação de pagamento, e as operações administrativas de CRUD sobre jogos e keys. O documento contempla também as integrações externas necessárias ao funcionamento do e-commerce, com destaque para o gateway de pagamento e o serviço de notificação por e-mail. Funcionalidades adicionais citadas no Documento de Visão, como cashback, carteira virtual e parcelamento, são tratadas como evoluções futuras e mencionadas apenas no que afeta a extensibilidade da arquitetura.

1.3 Definições, Acrônimos e Abreviações

Os principais termos empregados ao longo deste documento são definidos a seguir, de modo a uniformizar o vocabulário técnico entre as partes interessadas.

Termo	Definição
Key (chave)	Código alfanumérico de ativação que habilita o uso de um jogo digital em uma plataforma de distribuição (Steam, Epic Games Store, Origin, entre outras). Constitui o produto efetivamente comercializado pelo Cloud Key.

DRM	Digital Rights Management (Gestão de Direitos Digitais). Conjunto de tecnologias que controlam o uso e a ativação de conteúdos digitais; no contexto do Cloud Key, indica em qual plataforma a key deve ser resgatada.
DRM-free	Conteúdo digital distribuído sem mecanismos de gestão de direitos, que pode ser baixado e executado sem vínculo obrigatório a uma plataforma específica.
QoS	Quality of Service (Qualidade de Serviço). Conjunto de atributos que descrevem requisitos não funcionais do sistema, tais como desempenho, disponibilidade, escalabilidade, confiabilidade e segurança.
MVC	Model-View-Controller. Padrão arquitetural que separa responsabilidades em três camadas: o Model (dados e regras de negócio), a View (apresentação) e o Controller (orquestração entre ambos).
MER	Modelo Entidade-Relacionamento. Representação conceitual das entidades de dados, seus atributos e relacionamentos, base para o projeto do banco de dados relacional.
CRUD	Create, Read, Update, Delete. Conjunto das quatro operações básicas de persistência aplicadas às entidades do sistema, como jogos e keys.
API	Application Programming Interface. Interface de comunicação por meio da qual o Cloud Key integra-se a serviços externos, como o gateway de pagamento.
Gateway de pagamento	Serviço externo responsável por processar transações financeiras (cartão de crédito, Pix e outros meios), autorizar pagamentos e apoiar políticas antifraude.
HU	História de Usuário. Descrição sucinta de uma funcionalidade sob a perspectiva de um ator, utilizada como insumo para a derivação dos casos de uso.
ORM	Object-Relational Mapping. Mecanismo que mapeia objetos da aplicação para tabelas do banco de dados relacional, simplificando o acesso à persistência.
DAS	Documento de Arquitetura de Software, designação do presente documento.

Tabela 1 — Definições, acrônimos e abreviações

1.4 Referências

As fontes a seguir foram utilizadas como base para a elaboração deste documento ou são relevantes para a compreensão das decisões arquiteturais nele descritas.

- **[VISA0]** Documento de Visão do Cloud Key — Grupo 8, UCB, 2026. Contém a introdução ao problema, o contexto de negócio, o posicionamento, a descrição das partes interessadas e os requisitos não funcionais preliminares.
- **[APPWEB]** Documentação da Aplicação Web Cloud Key — Grupo 8, UCB, 2026. Apresenta as histórias de usuário, o projeto de interface, as funcionalidades de cadastro, autenticação e CRUD, a organização do código em MVC e o Modelo Entidade-Relacionamento.
- **[MDS]** Modelo de Documento de Arquitetura de Software (Metodologia de Desenvolvimento de Software), utilizado como gabarito de estrutura e seções para o presente documento.

- **[KRU41]** KRUCHTEN, Philippe. The “4+1” View Model of Software Architecture. IEEE Software, novembro de 1995.
- **[QOS]** Documentação de referência sobre atributos de Quality of Service em sistemas de software (Oracle Technology Documentation).
- **[NODE]** Documentação oficial do Node.js e do framework Express, base tecnológica do backend.

2. REPRESENTAÇÃO ARQUITETURAL

A arquitetura do Cloud Key é descrita segundo o modelo de visões “4+1” de Kruchten [KRU41], que organiza a representação de um sistema em quatro visões complementares — Lógica, de Processos, de Implementação e de Implantação — coordenadas por uma quinta visão central, a de Casos de Uso. Cada visão captura um conjunto específico de preocupações e é endereçada a um público distinto, de modo que diferentes partes interessadas possam compreender o sistema a partir da perspectiva que lhes é mais relevante.

No contexto do Cloud Key, a visão Lógica descreve a organização do sistema em camadas e pacotes segundo o padrão MVC, evidenciando as entidades de domínio (Usuário, Jogo, Key, Pedido e ItemPedido) e suas responsabilidades. A visão de Processos trata dos aspectos dinâmicos e de desempenho, particularmente a forma como o sistema lida com requisições concorrentes durante picos de acesso, como em grandes campanhas promocionais. A visão de Implementação detalha a estrutura concreta do código-fonte — a organização de pastas em models, controllers, routes e views da stack Node.js/Express — e a realização dos casos de uso mais significativos por meio de diagramas de classes e de sequência. A visão de Implantação descreve os nós físicos e os ambientes nos quais os componentes são distribuídos, incluindo o servidor de aplicação, o servidor de banco de dados e as integrações externas. Por fim, a visão de Casos de Uso reúne os requisitos funcionais centrais que conduzem e validam as demais visões.

Adicionalmente, dada a relevância da persistência para um e-commerce de chaves digitais, este documento destaca também uma perspectiva de Dados, derivada do Modelo Entidade-Relacionamento, que orienta os especialistas e administradores responsáveis pelo armazenamento das informações de usuários, catálogo, estoque de keys e pedidos.

A tabela a seguir relaciona cada visão ao seu público-alvo, à área de preocupação atendida e aos principais artefatos previstos para o Cloud Key.

Visão	Público	Área de preocupação	Principais artefatos do Cloud Key
Lógica	Analistas e arquitetos	Realização dos casos de uso; organização em camadas e pacotes	Diagrama de camadas MVC; diagrama de pacotes (usuários, catálogo, pedidos, keys, pagamento)
Processos	Integradores	Desempenho, escalabilidade e concorrência	Estratégias de cache, balanceamento e tratamento de picos de acesso
Implementação	Programadores	Componentes e estrutura do código	Estrutura de pastas Node.js/Express; diagrama de classes e de sequência
Implantação	Operações e gerência de configuração	Nós físicos e ambientes	Diagrama de implantação (servidor web/app, servidor de BD, gateway)
Casos de Uso	Todos os interessados	Requisitos funcionais	Diagrama de casos de uso significativos e atores

Dados	Especialistas e administradores de dados	Persistência das informações	Modelo Entidade-Relacionamento (MER)
--------------	--	------------------------------	--------------------------------------

Tabela 2 — Visões, público, área de preocupação e artefatos do Cloud Key

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e as restrições que exercem impacto significativo sobre a arquitetura do Cloud Key. Tais requisitos derivam diretamente das necessidades de negócio e dos requisitos não funcionais preliminares estabelecidos no Documento de Visão, em especial a exigência de compra segura de chaves oficiais, a tolerância a picos de acesso durante promoções, a alta disponibilidade do ambiente de vendas e a extensibilidade para acomodar novas categorias de produtos. Para cada requisito ou restrição, indica-se a descrição correspondente e a solução arquitetural adotada.

A tabela a seguir consolida essas decisões. Sempre que o Documento de Visão não especifica explicitamente uma escolha tecnológica, a solução é assinalada como assunção de projeto, mantida coerente com a stack indicada na documentação da aplicação web.

Requisito / Restrição	Descrição	Solução arquitetural adotada
Linguagem e frameworks	Tecnologias de desenvolvimento do backend e do frontend.	Backend em JavaScript sobre Node.js com o framework Express; frontend em HTML, CSS e JavaScript, com renderização de páginas por meio de templates EJS. A organização do código segue o padrão MVC.
Plataforma de aplicação	Servidor de aplicação e ambiente de hospedagem.	Servidor Node.js executando a aplicação Express, hospedado em um provedor de nuvem ou serviço de aplicação. Assunção de projeto: hospedagem em um App Service / contêiner em nuvem, com proxy reverso responsável por TLS.
Persistência	Mecanismo de armazenamento das entidades (Usuário, Jogo, Key, Pedido, ItemPedido).	Banco de dados relacional, em virtude das relações 1:N e das chaves estrangeiras definidas no MER. Assunção de projeto: utilização de MySQL ou PostgreSQL, com acesso por meio de um ORM (por exemplo, Sequelize) na camada de Model.
Segurança	Autenticação, autorização, proteção de dados sensíveis e prevenção a fraudes, conforme exigido para um e-commerce de chaves.	Autenticação por e-mail e senha, com senhas armazenadas exclusivamente como hash. Sessões autenticadas (ou tokens) protegendo rotas restritas. Autorização baseada no tipo de usuário (cliente ou administrador). Tráfego sobre HTTPS, criptografia de dados sensíveis e processo de análise adicional para pedidos suspeitos antes da liberação da chave.
Integração de pagamento	Processamento de transações com cartão, Pix e parcelamento.	Integração com gateway de pagamento externo via API, isolada em um serviço dedicado na camada de aplicação, de forma a permitir a substituição do provedor sem impacto nas regras de negócio.

Disponibilidade	Continuidade de acesso ao catálogo e à biblioteca do usuário.	Arquitetura em camadas que favorece a implantação redundante do servidor de aplicação e o monitoramento contínuo; registro de falhas e notificação à equipe de operações em caso de problemas na liberação de chaves ou no pagamento.
Escalabilidade e desempenho	Suporte a picos de acesso durante grandes promoções sem degradação perceptível.	Aplicação preferencialmente sem estado de sessão acoplado ao processo, permitindo escalabilidade horizontal atrás de um balanceador; uso de cache para o catálogo e otimização de consultas ao banco por meio de índices.
Internacionalização (i18n)	Adequação de idioma e localização.	Interface concebida em português do Brasil, idioma-alvo do público. A separação de textos da camada de View facilita futura internacionalização, ainda que não seja requisito imediato.
Manutenibilidade e extensibilidade	Facilidade de correções e inclusão de novas categorias de produtos (por exemplo, gift cards).	Separação clara de responsabilidades em camadas MVC e em pacotes de domínio, reduzindo o acoplamento e permitindo a adição de novos módulos com impacto localizado.
Usabilidade e responsividade	Layout claro, navegação intuitiva e suporte a desktop e mobile.	Interface responsiva na camada de apresentação, com telas prototipadas em Figma, links íntegros e feedback claro de erros ao usuário.

Tabela 3 — Requisitos e restrições arquiteturais do Cloud Key

4. VISÃO DE CASOS DE USO

Esta seção apresenta os casos de uso centrais e significativos para a arquitetura do Cloud Key, derivados das histórias de usuário e dos requisitos consolidados na documentação da aplicação web. A seleção privilegia os casos de uso que exercitam mecanismos arquiteturais relevantes, tais como autenticação, autorização, persistência, integração com serviços externos e as regras de negócio centrais do e-commerce.

4.1 Casos de Uso significantes para a arquitetura

Os principais atores que interagem com o sistema são o Jogador/Cliente, o Administrador da Loja e dois atores externos: o Sistema de Pagamento (gateway) e o Serviço de E-mail responsável por notificações. O Jogador/Cliente representa o usuário final que cria sua conta, navega pelo catálogo e adquire chaves; o Administrador da Loja gerencia o catálogo, o estoque de keys e, opcionalmente, os usuários; o Sistema de Pagamento processa e autoriza as transações financeiras, condicionando a liberação das chaves; e o Serviço de E-mail entrega confirmações e chaves ao cliente.

A lista a seguir descreve os casos de uso significativos para a arquitetura, indicando, em cada caso, por que ele é arquiteturalmente relevante.

Código	Caso de uso	Ator principal	Relevância arquitetural
CU001	Cadastrar cliente	Jogador/Cliente	Exercita validação de dados e o armazenamento seguro de senhas (hash), inaugurando o fluxo de identidade do sistema.
CU002	Autenticar (login/logout)	Jogador/Cliente; Administrador	Estabelece o mecanismo de sessão/token e a proteção de rotas restritas, base para a autorização de todo o sistema.
CU003	Visualizar catálogo de jogos	Jogador/Cliente	É o caso de uso de maior volume de acesso; influencia diretamente as decisões de cache e de otimização de consultas para suportar picos.
CU004	Comprar key de jogo	Jogador/Cliente; Sistema de Pagamento	Caso de uso central: integra autenticação, carrinho, persistência transacional, integração externa de pagamento e a regra de geração e associação da chave. É a realização detalhada na Visão de Implementação.
CU005	Gerenciar catálogo (CRUD de jogos/keys)	Administrador	Exercita autorização administrativa e o controle de estoque de keys, com impacto sobre a integridade dos dados de venda.
CU006	Gerenciar usuários	Administrador	Caso de uso opcional que reforça as necessidades de autorização e de segregação de perfis.
CU007	Consultar histórico e visualizar chave adquirida	Jogador/Cliente	Exercita a confirmação de segurança para exibição da key e a recuperação de pedidos persistidos,

			demandando controle de acesso refinado.
--	--	--	---

Tabela 4 — Casos de uso significativos para a arquitetura do Cloud Key

Entre os casos de uso listados, o CU004 — Comprar key de jogo — é o que melhor sintetiza as preocupações arquiteturais do Cloud Key, pois percorre praticamente todas as camadas e mecanismos do sistema: parte da interface autenticada, transita pelo controlador de pedidos, aciona o serviço de pagamento externo, persiste o pedido e seus itens de forma transacional, atualiza o estoque de keys e, somente após a confirmação do pagamento, associa a chave à conta do cliente. Por essa razão, ele é adotado como caso de uso de referência na Visão de Implementação.

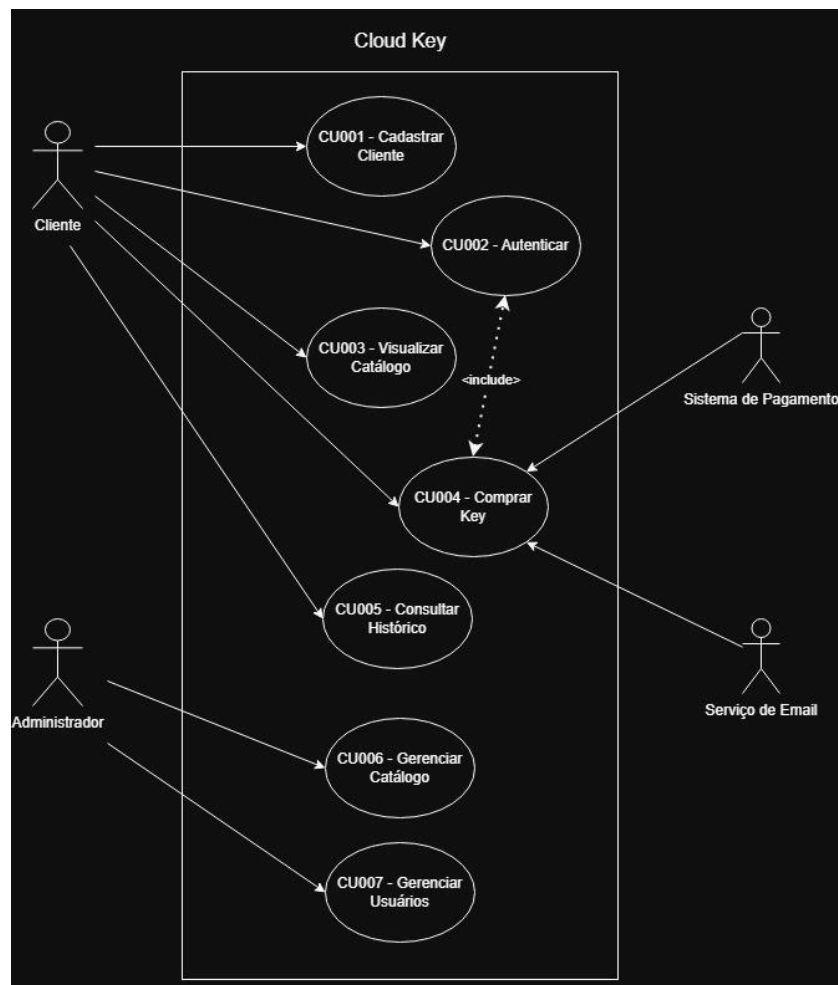


Figura 1 — Casos de uso significativos do Cloud Key

5. VISÃO LÓGICA

A visão lógica descreve a organização interna do Cloud Key em camadas e pacotes, evidenciando as classes e os subsistemas mais significativos e a forma como se relacionam. A arquitetura adota o padrão Model-View-Controller (MVC) indicado na documentação da aplicação, refinado em uma estrutura de quatro camadas lógicas que separa de maneira nítida a apresentação, a orquestração de aplicação, as regras de domínio e o acesso à persistência.

5.1 Visão Geral — pacotes e camadas

A camada de Apresentação corresponde à View do padrão MVC e é responsável exclusivamente por exibir informações ao usuário e capturar suas interações. É composta pelos templates renderizados no servidor (login, cadastro, catálogo, detalhe do jogo, carrinho e painel administrativo) e pelos recursos estáticos de estilo e script. Esta camada não contém regra de negócio nem acessa diretamente o banco de dados; recebe dados já preparados pelos controladores e os apresenta de forma responsiva.

A camada de Aplicação/Controle abriga os controladores e as rotas. Os controladores recebem as requisições encaminhadas pelas rotas do Express, coordenam a execução dos serviços e dos modelos de domínio, decidem qual View será renderizada e quais dados a ela serão enviados. É também nessa camada que residem os middlewares de autenticação e autorização, que verificam se o usuário está logado e se possui o perfil necessário antes de permitir o acesso a rotas restritas, como o carrinho, o histórico de compras e o painel administrativo.

A camada de Domínio/Negócio concentra as entidades centrais do sistema e as regras que as governam. Nela situam-se os serviços de negócio — por exemplo, o serviço de pedidos, que coordena a compra; o serviço de pagamento, que encapsula a comunicação com o gateway; e o serviço de chaves, que controla a geração e a associação de keys. As entidades de domínio Usuário, Jogo, Key, Pedido e ItemPedido representam os conceitos essenciais do e-commerce e materializam regras como a verificação de disponibilidade de estoque de chaves antes de confirmar uma compra.

A camada de Persistência/Infraestrutura encapsula o acesso ao banco de dados relacional e às integrações externas. Os modelos (models) e seus repositórios traduzem as operações de domínio em consultas, inserções, atualizações e exclusões sobre as tabelas, preferencialmente por meio de um ORM. Esta camada também isola os adaptadores de serviços externos, como o cliente HTTP do gateway de pagamento e o serviço de envio de e-mails, de modo que mudanças de provedor não afetem as camadas superiores.

As dependências entre as camadas obedecem a um sentido único e descendente: a Apresentação depende da Aplicação/Controle, que depende do Domínio, que por sua vez depende da Persistência/Infraestrutura. Esse encadeamento reduz o acoplamento e favorece a manutenibilidade e a testabilidade, atributos exigidos pelo Documento de Visão ao prever a evolução do catálogo e a integração futura de novos serviços.

Internamente, o domínio é subdividido em pacotes funcionais coesos. O pacote de usuários e autenticação reúne o cadastro, o login e o controle de perfis. O pacote de catálogo de jogos cuida da consulta e da gestão dos títulos. O pacote de keys administra o estoque de chaves e seus estados (disponível, vendida, expirada). O pacote de pedidos coordena o

carrinho, a finalização da compra e o histórico. E o pacote de pagamento isola a integração financeira. Essa modularização permite que cada pacote evolua de forma relativamente independente.



Figura 2 — Diagrama de pacotes e camadas lógicas do Cloud Key

O modelo de dados conecta-se à camada de domínio de maneira direta: cada entidade conceitual do MER possui uma classe de domínio e um modelo de persistência correspondentes. Um Usuário realiza muitos Pedidos; cada Pedido agrega muitos

ItemPedido; cada ItemPedido referencia um Jogo; e cada Jogo possui muitas Keys, das quais uma é vinculada ao item de pedido no momento da venda. Essas associações orientam tanto o desenho das classes de domínio quanto o mapeamento objeto-relacional realizado na camada de persistência.

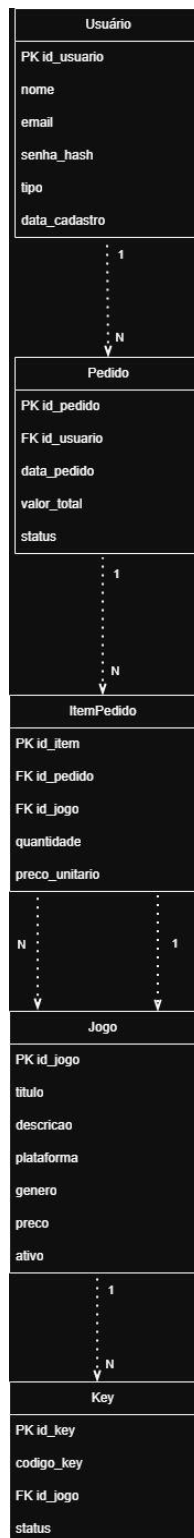


Figura 3 — Modelo Entidade-Relacionamento (MER) do Cloud Key

6. VISÃO DE IMPLEMENTAÇÃO

A visão de implementação descreve a estrutura concreta do código-fonte do Cloud Key e a realização de um caso de uso significativo por meio da colaboração entre suas classes. Em conformidade com a documentação da aplicação, o projeto adota a stack Node.js com Express e organiza os arquivos segundo o padrão MVC, refletindo no sistema de arquivos a estrutura lógica descrita na seção anterior.

A organização de pastas reflete diretamente as camadas: o diretório **/src/models** contém os modelos de domínio e persistência (`usuarioModel`, `jogoModel`, `pedidoModel` e o modelo de `key`); **/src/controllers** reúne os controladores (`authController`, `usuarioController`, `jogoController` e `pedidoController`); **/src/routes** define as rotas que mapeiam URLs para ações dos controladores; e **/src/views** abriga os templates EJS de cada tela. O diretório **/public** disponibiliza os recursos estáticos de CSS, JavaScript e imagens, enquanto o arquivo **app.js** inicializa a aplicação Express e registra as rotas e middlewares. Dessa forma, a separação física de responsabilidades torna evidente a correspondência entre código e arquitetura.

Os componentes de frontend e backend relacionam-se de modo que cada rota encaminha a requisição ao controlador apropriado; o controlador aciona os modelos e serviços de domínio e, ao final, seleciona o template EJS a ser renderizado com os dados obtidos. As páginas resultantes, servidas ao navegador, podem conter scripts auxiliares para validações simples e interatividade, mantendo, porém, toda a regra de negócio no servidor.

6.1 Caso de Uso CU004 — Comprar key de jogo

O caso de uso Comprar key de jogo é detalhado por exercitar os mecanismos arquiteturais mais críticos do Cloud Key. Ele inicia com um cliente autenticado que possui itens no carrinho e finaliza com a chave de ativação associada à sua conta, após a confirmação do pagamento pelo gateway externo.

6.1.1 Diagrama de Classes

A realização do CU004 envolve um conjunto coeso de classes distribuídas pelas camadas do sistema. Na camada de controle, o `PedidoController` recebe a requisição de finalização da compra encaminhada por uma rota protegida e orquestra a execução, retornando ao final a View de confirmação. Ele depende do `PedidoService`, classe da camada de domínio responsável por coordenar a transação de compra: validar o carrinho, verificar a disponibilidade de keys, calcular o valor total, registrar o pedido e seus itens e acionar o pagamento.

O `PedidoService` colabora com o `PagamentoService`, que encapsula a comunicação com o gateway externo e devolve o resultado da autorização, e com o `KeyService`, responsável por selecionar uma chave disponível para cada jogo adquirido e por alterar seu estado para vendida no momento da confirmação. O acesso aos dados é mediado pelos repositórios `JogoRepository`, `PedidoRepository` e `KeyRepository`, que traduzem as operações em comandos sobre as respectivas tabelas.

As entidades de domínio participantes são `Pedido`, que agrega uma coleção de `ItemPedido`; `ItemPedido`, que referencia um `Jogo` e, quando concluída a venda, vincula-se a uma `Key`; e `Usuário`, proprietário do pedido. As relações entre essas classes refletem o MER: a

associação de composição entre Pedido e ItemPedido (um item não existe sem o pedido que o contém), a associação de cada ItemPedido a um único Jogo, e a associação de cada Jogo a muitas Keys, das quais uma é atribuída ao item no ato da venda. As dependências entre serviços e repositórios são de uso (o serviço utiliza o repositório), sem herança, favorecendo o baixo acoplamento.

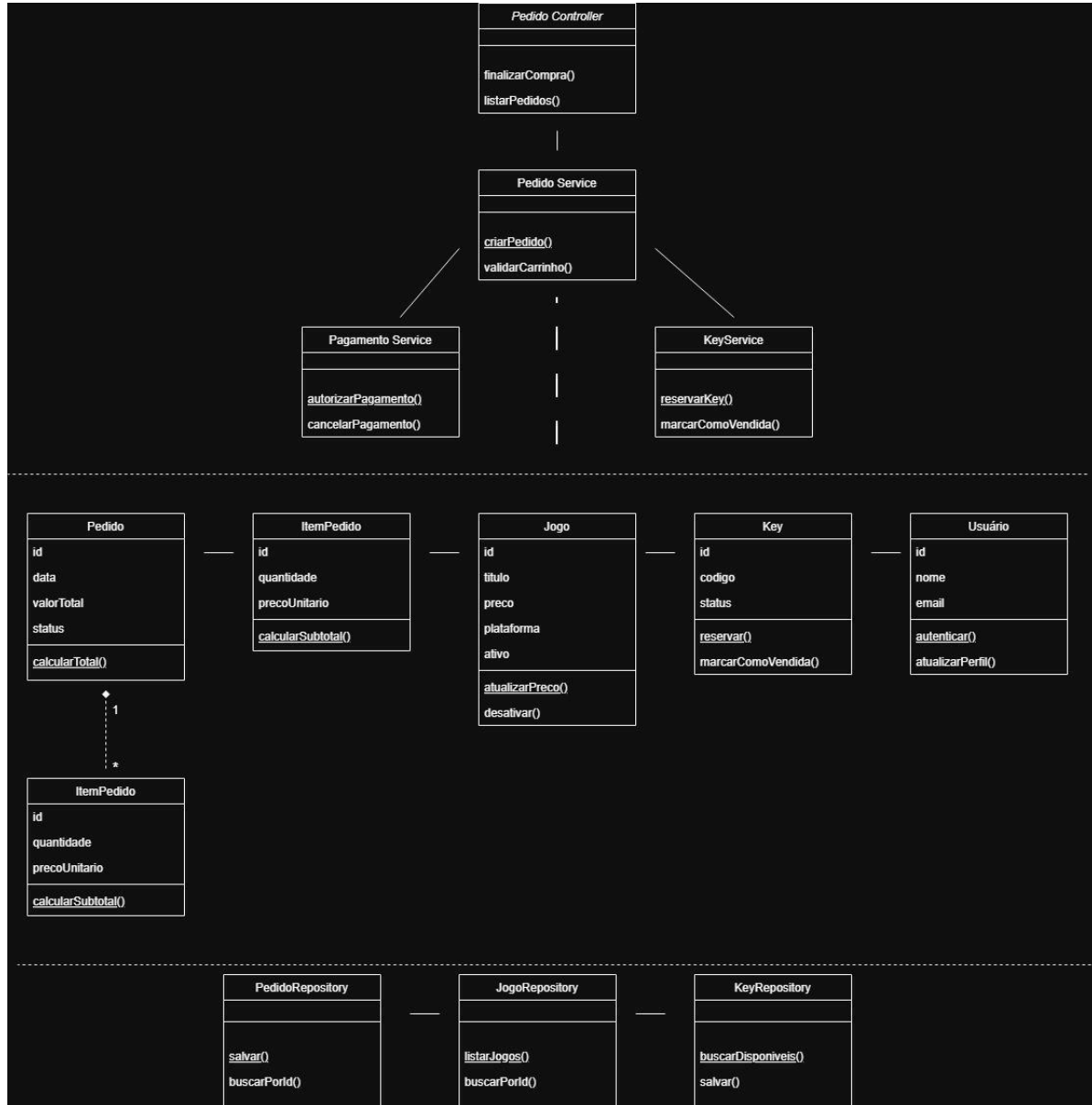


Figura 4 — Diagrama de classes do caso de uso CU004 — Comprar key de jogo

6.1.2 Diagrama de Sequência

O fluxo de mensagens do CU004 percorre as camadas na seguinte ordem. O cliente, na tela de carrinho/checkout, aciona a opção “Finalizar compra”, o que dispara uma requisição HTTP à rota protegida de pedidos. O middleware de autenticação verifica a sessão do usuário e, validada a identidade, encaminha a requisição ao PedidoController.

1. O PedidoController recebe a requisição de checkout e solicita ao PedidoService a criação do pedido a partir do carrinho do usuário autenticado.

2. O PedidoService consulta o JogoRepository e o KeyRepository para confirmar que há keys disponíveis para cada jogo do carrinho e para obter os preços atualizados, calculando o valor total.
3. Com o pedido validado, o PedidoService aciona o PagamentoService, que envia a requisição de cobrança ao gateway de pagamento externo e aguarda a resposta de autorização.
4. Recebida a autorização positiva, o PedidoService persiste o Pedido e os respectivos ItemPedido por meio do PedidoRepository, em uma transação única, garantindo a consistência dos dados.
5. O PedidoService solicita ao KeyService a reserva de uma Key disponível para cada item; o KeyService atualiza o estado das chaves para venda via KeyRepository e as vincula aos itens do pedido.
6. O PedidoService atualiza o status do pedido para concluído e, opcionalmente, solicita ao serviço de e-mail o envio da confirmação e da chave ao cliente.
7. O controle retorna ao PedidoController, que renderiza a View de confirmação, exibindo o resumo do pedido e a chave adquirida — esta última condicionada à confirmação de segurança prevista no fluxo de pós-venda.

Caso o gateway recuse o pagamento ou ocorra falha na liberação da chave, o fluxo é interrompido antes da conclusão: o sistema registra o ocorrido, mantém o pedido em estado de processamento ou cancelado, notifica a equipe de operações e informa o usuário, em conformidade com o requisito não funcional de tratamento de exceções descrito no Documento de Visão.

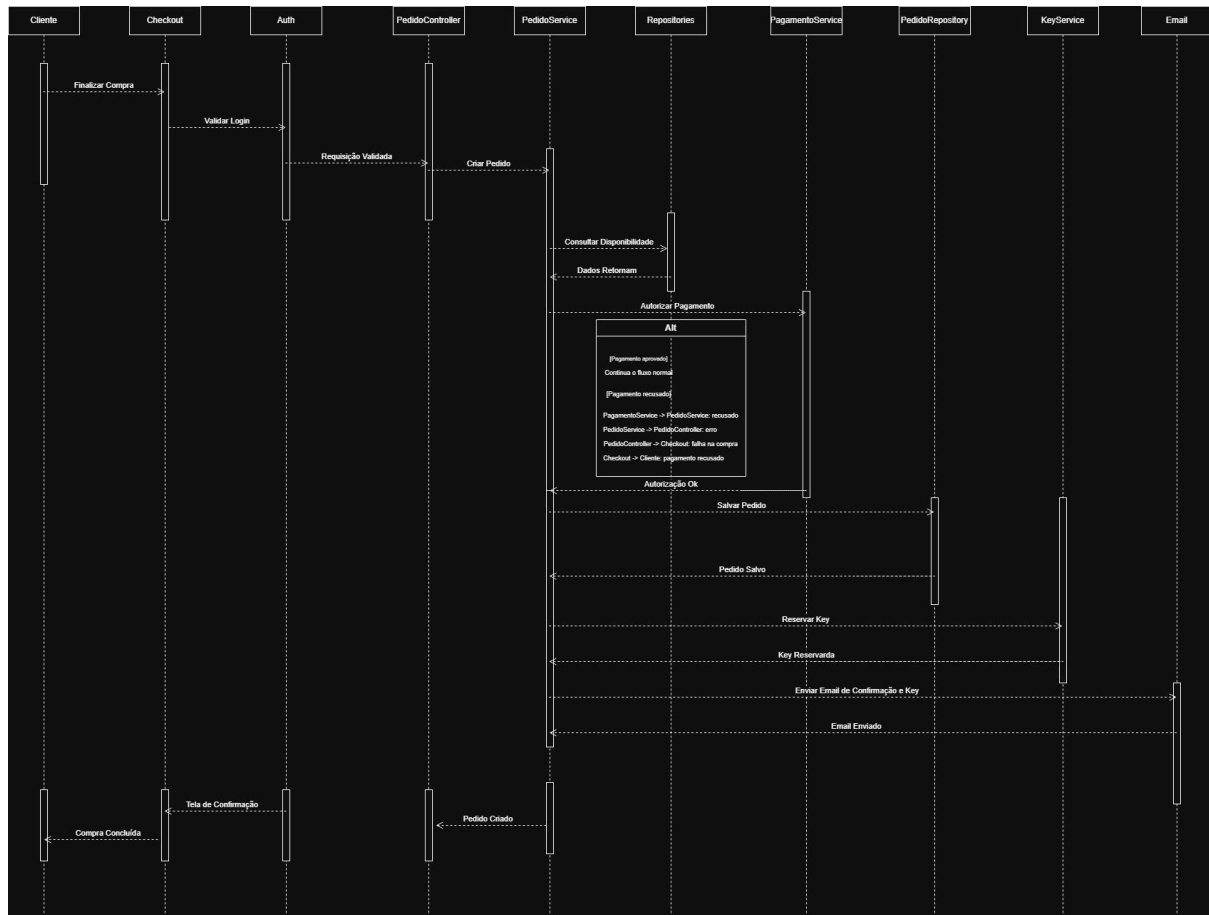


Figura 5 — Diagrama de sequência do caso de uso CU004 — Comprar key de jogo

7. VISÃO DE IMPLANTAÇÃO

A visão de implantação descreve os nós físicos e os ambientes nos quais os componentes lógicos do Cloud Key são distribuídos, bem como a comunicação entre eles e as dependências externas. Considerando o caráter acadêmico do projeto e os requisitos de disponibilidade e escalabilidade do Documento de Visão, prevê-se uma topologia simples, porém preparada para crescimento.

Estão previstos três ambientes ao longo do ciclo de vida: o ambiente de desenvolvimento, no qual a equipe executa a aplicação localmente com um banco de dados de teste; o ambiente de homologação, destinado à validação de funcionalidades antes da liberação; e o ambiente de produção, que atende aos usuários finais. Os três ambientes compartilham a mesma arquitetura lógica, diferindo apenas em escala e em parâmetros de configuração.

Em produção, o cliente acessa o sistema por meio de um navegador (desktop ou mobile), que se comunica com o **nó de aplicação** através do protocolo HTTPS. Esse nó hospeda a aplicação Node.js/Express, responsável por servir as páginas renderizadas e por executar a lógica de controle e de domínio; nele residem, portanto, as camadas de Apresentação, Aplicação/Controle e Domínio. Assunção de projeto: o nó de aplicação opera atrás de um proxy reverso que encerra o TLS e pode atuar como balanceador de carga, permitindo a execução de múltiplas instâncias da aplicação para suportar picos de acesso.

A camada de Persistência reside em um **nó de banco de dados** dedicado, que executa o sistema gerenciador de banco de dados relacional e armazena as tabelas de usuários, jogos, keys, pedidos e itens de pedido. A comunicação entre o nó de aplicação e o nó de banco ocorre por conexão segura na rede interna do ambiente. Manter o banco em um nó separado favorece a escalabilidade independente e o isolamento dos dados sensíveis.

Como dependências externas, o nó de aplicação integra-se ao gateway de pagamento e ao serviço de e-mail, ambos acessados por chamadas HTTPS a APIs de terceiros. O gateway de pagamento processa as transações e devolve as autorizações que condicionam a liberação das chaves, enquanto o serviço de e-mail entrega confirmações e chaves aos clientes. O isolamento dessas integrações em adaptadores específicos da camada de infraestrutura permite substituir provedores sem afetar os demais nós.

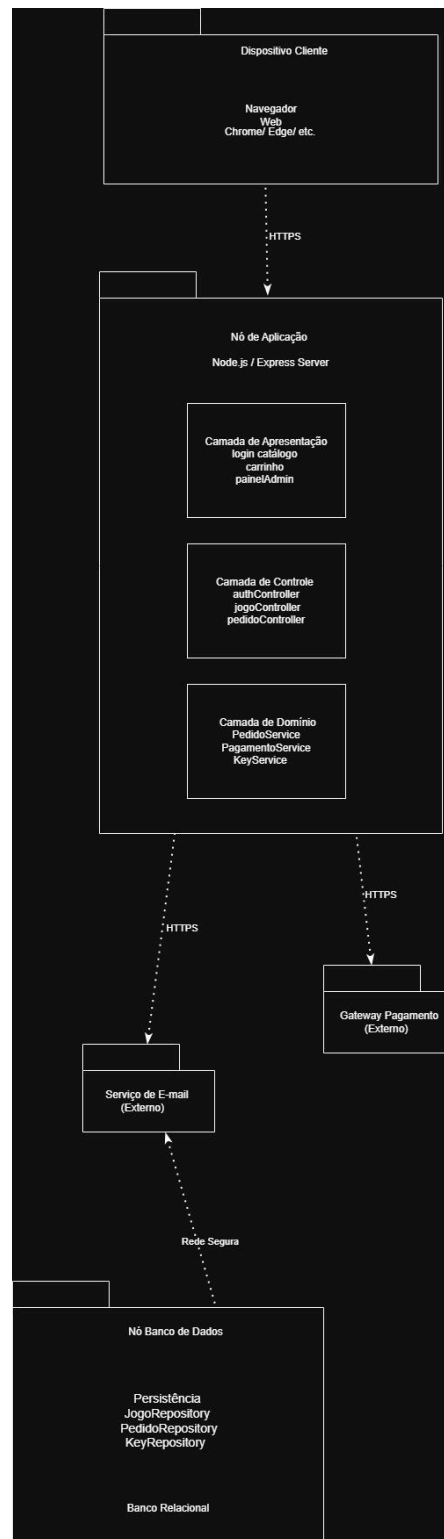


Figura 6 — Diagrama de implantação do Cloud Key

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Esta seção apresenta estimativas de volume de uso do Cloud Key. Como o Documento de Visão não fornece números absolutos, as estimativas a seguir são dimensionadas para um e-commerce acadêmico de porte pequeno a médio, com folga para crescimento durante campanhas promocionais inspiradas nas práticas da Nuuvem.

Assunção de projeto: estima-se uma base inicial de aproximadamente 5.000 usuários cadastrados, com cerca de 1.000 a 2.000 acessos diários em operação normal.

Assunção de projeto: durante grandes promoções, o número de acessos diários pode atingir de 5 a 10 vezes o volume habitual, alcançando picos da ordem de 15.000 a 20.000 acessos concentrados em poucas horas.

Assunção de projeto: estima-se de 100 a 300 transações de compra por dia em operação normal, podendo ultrapassar 1.000 transações diárias em períodos de pico, com forte concentração temporal.

Assunção de projeto: o catálogo inicial é projetado para algo entre 500 e 2.000 títulos, com um estoque de keys que pode chegar a algumas dezenas de milhares de registros, dado que cada jogo possui múltiplas chaves.

Assunção de projeto: o tempo médio de sessão de um usuário situa-se em torno de 8 a 15 minutos, concentrado na navegação pelo catálogo e na consulta a detalhes de jogos.

Esses números orientam o dimensionamento dos recursos de infraestrutura e justificam as estratégias de cache e de escalabilidade horizontal previstas, de modo que o sistema absorva os picos sem degradação perceptível pelo usuário.

8.2 Performance

As expectativas de desempenho do Cloud Key visam preservar a experiência do usuário mesmo sob carga elevada. Estabelecem-se as seguintes metas de tempo de resposta, assinaladas como assunções de projeto por não constarem numericamente no Documento de Visão.

Assunção de projeto: o carregamento da página de catálogo deve ocorrer em até 2 segundos em condições normais e não ultrapassar 4 segundos durante picos de acesso.

Assunção de projeto: a conclusão de uma compra, desde o acionamento de “Finalizar compra” até a exibição da confirmação, deve ocorrer em até 5 segundos, incluindo o tempo de resposta do gateway de pagamento.

Para atingir essas metas e lidar com promoções e picos de acesso, a arquitetura prevê o uso de cache para os dados de catálogo, que são predominantemente de leitura, reduzindo a pressão sobre o banco de dados. A escalabilidade horizontal do nó de aplicação, viabilizada pelo desacoplamento do estado de sessão e pelo balanceamento de carga, permite acrescentar instâncias durante campanhas. Adicionalmente, a otimização das consultas mais frequentes por meio de índices apropriados sobre as tabelas de jogos e keys, a paginação das listagens e o uso eventual de escalabilidade vertical no banco de

dados completam o conjunto de estratégias adotadas para sustentar o desempenho esperado.

9. QUALIDADE

Esta seção enumera os atributos de qualidade de serviço (QoS) mais significativos para o Cloud Key, relacionando, para cada um, a meta ou expectativa associada e as decisões arquiteturais que contribuem para alcançá-la. Os atributos derivam dos requisitos não funcionais consolidados no Documento de Visão.

Atributo de QoS	Meta / expectativa	Decisões arquiteturais de apoio
Disponibilidade	Acesso contínuo ao catálogo e à biblioteca do usuário, evitando indisponibilidades prolongadas, sobretudo no horário de maior movimento.	Arquitetura em camadas que permite implantação redundante do nó de aplicação; monitoramento contínuo; registro de falhas e notificação automática à equipe de operações.
Escalabilidade	Absorver picos de acesso em grandes promoções sem degradação perceptível.	Aplicação sem estado acoplado ao processo, escalabilidade horizontal atrás de balanceador, cache do catálogo e otimização de consultas.
Segurança	Proteger dados sensíveis, autenticar usuários e prevenir fraudes antes da liberação de chaves.	Senhas armazenadas como hash; comunicação sobre HTTPS; autorização por perfil; isolamento do gateway de pagamento; análise adicional de pedidos suspeitos; confirmação de segurança para exibir a chave.
Confiabilidade	Garantir a consistência das compras e evitar venda de chaves indisponíveis ou duplicadas.	Persistência transacional do pedido e seus itens; verificação de estoque de keys antes da confirmação; controle de estado das chaves (disponível, vendida, expirada).
Manutenibilidade	Permitir correções e a inclusão de novas categorias de produtos e serviços com impacto localizado.	Separação clara em camadas MVC e pacotes de domínio coesos; baixo acoplamento; isolamento de integrações externas em adaptadores.
Usabilidade	Navegação intuitiva e layout claro em desktop e mobile, com feedback adequado de erros.	Interface responsiva na camada de apresentação; telas prototipadas em Figma; mensagens de validação e de erro consistentes.
Desempenho	Tempos de resposta baixos no carregamento do catálogo e na conclusão da compra.	Cache de leitura, paginação, índices nas tabelas mais consultadas e dimensionamento elástico do nó de aplicação.
Portabilidade	Facilitar a implantação nos diferentes ambientes (desenvolvimento, homologação e produção).	Stack baseada em Node.js, independente de sistema operacional; configuração externalizada por ambiente; possibilidade de empacotamento em contêineres.

Tabela 5 — Atributos de qualidade (QoS) do Cloud Key

Guia para elaboração dos diagramas

Esta seção final orienta a equipe na construção manual dos diagramas previstos para o Documento de Arquitetura, em ferramentas como draw.io, Lucidchart ou brModelo. Não são apresentados os diagramas em si; fornece-se, para cada um, a relação dos elementos que devem aparecer e um passo a passo resumido para a montagem a partir das descrições textuais deste documento.

1. Diagrama de Casos de Uso significativo

Título sugerido: “Casos de uso principais do Cloud Key”.

Atores: Jogador/Cliente e Administrador da Loja (atores humanos, à esquerda); Sistema de Pagamento e Serviço de E-mail (atores externos, à direita).

Casos de uso: Cadastrar cliente (CU001), Autenticar (CU002), Visualizar catálogo de jogos (CU003), Comprar key de jogo (CU004), Gerenciar catálogo de jogos e keys (CU005), Gerenciar usuários (CU006) e Consultar histórico e visualizar chave (CU007).

1. Desenhe um retângulo grande representando a fronteira do sistema e escreva “Cloud Key” no topo.
2. Posicione o Jogador/Cliente e o Administrador como bonecos (stick figures) à esquerda da fronteira; posicione o Sistema de Pagamento e o Serviço de E-mail à direita.
3. Coloque dentro da fronteira as elipses dos casos de uso; agrupe à esquerda os do cliente (CU001 a CU004 e CU007) e mais abaixo os administrativos (CU005 e CU006).
4. Trace associações (linhas simples) entre o Jogador/Cliente e CU001, CU002, CU003, CU004 e CU007; entre o Administrador e CU002, CU005 e CU006.
5. Ligue o Sistema de Pagamento ao CU004 e o Serviço de E-mail ao CU004 (notificação da chave), por serem atores secundários desse caso de uso.
6. Se desejar, represente a relação «include» de CU004 para CU002, indicando que comprar exige estar autenticado.

2. Diagrama de Pacotes / Camadas da Visão Lógica

Título sugerido: “Camadas e pacotes lógicos do Cloud Key”.

Camadas (de cima para baixo): Apresentação (Views/EJS e recursos estáticos); Aplicação/Controle (Controllers, Routes e middlewares de autenticação); Domínio/Negócio (entidades e serviços); Persistência/Infraestrutura (Repositories/Models, ORM e adaptadores externos).

Exemplos por camada: Apresentação — login, catálogo, carrinho, painelAdmin; Controle — authController, jogoController, pedidoController; Domínio — entidades Usuário, Jogo, Key, Pedido, ItemPedido e serviços PedidoService, PagamentoService, KeyService; Persistência — JogoRepository, PedidoRepository, KeyRepository, adaptador do gateway e do e-mail.

1. Empilhe quatro retângulos horizontais, um para cada camada, na ordem indicada acima.

2. Dentro de cada camada, desenhe retângulos menores (pacotes) com exemplos de classes/componentes correspondentes.
3. Use setas tracejadas de dependência apontando de cima para baixo: Apresentação → Controle → Domínio → Persistência, sem setas no sentido inverso.
4. Opcionalmente, destaque os pacotes de domínio (usuários, catálogo, keys, pedidos, pagamento) como subdivisões da camada de Domínio.

3. Diagrama de Classes do CU004 — Comprar key de jogo

Classes: PedidoController; serviços PedidoService, PagamentoService e KeyService; repositórios JogoRepository, PedidoRepository e KeyRepository; entidades Pedido, ItemPedido, Jogo, Key e Usuário.

Atributos e métodos em alto nível: Pedido (id, data, valorTotal, status; calcularTotal()); ItemPedido (id, quantidade, precoUnitario); Jogo (id, titulo, preco, plataforma, ativo); Key (id, codigo, status); Usuário (id, nome, email); PedidoService (criarPedido(), validarCarrinho()); PagamentoService (autorizarPagamento()); KeyService (reservarKey(), marcarComoVendida()).

1. Crie um retângulo de classe para cada item, com três compartimentos: nome, atributos e métodos.
2. Desenhe uma composição (losango preenchido no lado do Pedido) entre Pedido e ItemPedido, com cardinalidade 1 para muitos.
3. Desenhe uma associação de ItemPedido para Jogo (muitos para um) e de ItemPedido para Key (um para um, quando vendida).
4. Ligue Usuário a Pedido por associação (um usuário realiza muitos pedidos) e Jogo a Key (um jogo possui muitas keys).
5. Use setas tracejadas de dependência de PedidoController para PedidoService, e deste para PagamentoService, KeyService e os repositórios; não utilize herança.

4. Diagrama de Sequência do CU004 — Comprar key de jogo

Participantes (linhas de vida, da esquerda para a direita): Cliente, Tela de Carrinho/Checkout, Middleware de Autenticação, PedidoController, PedidoService, JogoRepository/KeyRepository, PagamentoService (Gateway de Pagamento), PedidoRepository, KeyService e, opcionalmente, Serviço de E-mail.

1. Desenhe as linhas de vida verticais de cada participante, na ordem indicada.
2. Mensagem 1: Cliente → Tela de Carrinho: aciona “Finalizar compra”; em seguida a Tela envia a requisição HTTP ao Middleware de Autenticação.
3. Mensagem 2: Middleware → PedidoController: requisição validada; PedidoController → PedidoService: criarPedido().
4. Mensagem 3: PedidoService → JogoRepository/KeyRepository: consultar disponibilidade e preços (mensagens de retorno trazem os dados).
5. Mensagem 4: PedidoService → PagamentoService → Gateway: autorizarPagamento(); o retorno traz a autorização.
6. Mensagem 5: PedidoService → PedidoRepository: persistir Pedido e ItemPedido em transação.

7. Mensagem 6: PedidoService → KeyService → KeyRepository: reservar e marcar keys como vendidas, vinculando-as aos itens.
8. Mensagem 7 (opcional): PedidoService → Serviço de E-mail: enviar confirmação e chave.
9. Mensagem 8: retorno ao PedidoController, que renderiza a Tela de Confirmação para o Cliente. Represente com um fragmento alternativo (alt) o caminho de recusa do pagamento, que interrompe o fluxo e notifica as operações.

5. Diagrama de Implantação

Nós: Dispositivo do Cliente (navegador); Nó de Aplicação (Node.js/Express, atrás de proxy reverso/balanceador); Nó de Banco de Dados (SGBD relacional); Gateway de Pagamento (externo); Serviço de E-mail (externo).

1. Represente cada nó como um cubo (node) rotulado; coloque o Dispositivo do Cliente à esquerda e o Nó de Aplicação ao centro.
2. Dentro do Nó de Aplicação, indique os artefatos implantados: a aplicação Express com as camadas de Apresentação, Controle e Domínio.
3. Posicione o Nó de Banco de Dados abaixo ou ao lado do Nó de Aplicação e indique a camada de Persistência.
4. Trace a conexão Cliente → Aplicação rotulada como HTTPS; a conexão Aplicação → Banco como conexão segura na rede interna.
5. Coloque o Gateway de Pagamento e o Serviço de E-mail à direita, como nós externos, ligados ao Nó de Aplicação por conexões HTTPS rotuladas.

6. MER / Diagrama Entidade-Relacionamento

Entidades e atributos principais: Usuário (id_usuario PK, nome, email único, senha_hash, tipo, data_cadastro); Jogo (id_jogo PK, titulo, descricao, plataforma, genero, preco, ativo); Key (id_key PK, codigo_key, id_jogo FK, status); Pedido (id_pedido PK, id_usuario FK, data_pedido, valor_total, status); ItemPedido (id_item PK, id_pedido FK, id_jogo FK, quantidade, preco_unitario).

1. Desenhe um retângulo para cada entidade e liste seus atributos, sublinhando ou marcando as chaves primárias (PK) e identificando as chaves estrangeiras (FK).
2. Ligue Usuário a Pedido com cardinalidade 1:N (um usuário realiza muitos pedidos).
3. Ligue Pedido a ItemPedido com cardinalidade 1:N (um pedido possui muitos itens).
4. Ligue ItemPedido a Jogo com cardinalidade N:1 (cada item refere-se a um jogo).
5. Ligue Jogo a Key com cardinalidade 1:N (um jogo possui muitas keys).
6. Opcionalmente, ligue Key a ItemPedido com cardinalidade 1:1, representando a chave entregue quando o item é vendido.
7. Caso seja preciso modelar futuras relações N:M (por exemplo, jogos em múltiplas promoções), crie uma entidade associativa com as chaves estrangeiras das entidades relacionadas.